

Algoritmos y Estructuras de Datos II**TP Complejidad****Ejercicio 1:**

Demuestre que $6n^3 \neq O(n^2)$.

Por definición, $T(n)$ es $O(f(n))$ si existen constantes positivas c y n_0 tales que:

$$0 \leq T(n) \leq cf(n), \text{ para todo } n \geq n_0$$

Supongamos que $6n^3 = O(n^2)$. Entonces deberían existir constantes positivas c y n_0 tales que:

$$0 \leq 6n^3 \leq cn^2, \text{ para todo } n \geq n_0.$$

Como $n^2 > 0$, dividimos ambos lados por n^2 :

$$6n \leq c$$

Pero esto no puede cumplirse para todo $n \geq n_0$, ya que $6n$ aumenta indefinidamente a medida que n aumenta. Para cualquier constante positiva c , siempre podemos encontrar un valor de n suficientemente grande tal que $6n > c$.

Es por ello que no existen constantes positivas c y n_0 que cumplan la definición de $O(n^2)$.

Por lo tanto: $6n^3 \neq O(n^2)$.

Ejercicio 2:

¿Cómo sería un array de números (mínimo 10 elementos) para el mejor caso de la estrategia de ordenación Quicksort(n)?

El mejor caso de Quicksort se produce cuando el elemento elegido como pivote divide al array en dos partes de tamaños aproximadamente iguales en cada paso de la partición.

Por ejemplo, un array de 10 elementos que puede representar un mejor caso es:

[1,2,3,4,5,6,7,8,9,10]

si la implementación de Quicksort selecciona como pivote el elemento central. En este caso, el pivote divide el array en dos partes de tamaños similares, lo que permite obtener una complejidad de: $O(n \log n)$

Ejercicio 3:

¿Cuál es el tiempo de ejecución de la estrategia Quicksort(A), Insertion-Sort(A) y Merge-Sort(A) cuando todos los elementos del array A tienen el mismo valor?

- Quicksort(A): si todos los elementos tienen el mismo valor en la implementación usual el pivote no logra dividir el array en partes equilibradas, por lo que se produce el peor caso. Su tiempo de ejecución es $O(n^2)$.

- Insertion-Sort(A): como todos los elementos son iguales, no es necesario realizar desplazamientos. Igualmente, el algoritmo deberá recorrer los elementos del array, por lo que su tiempo de ejecución es $O(n)$.
- Merge-Sort(A): aunque todos los elementos sean iguales, el algoritmo continúa dividiendo el array y luego combinando las partes. Por lo tanto, su tiempo de ejecución es $O(n \log n)$.

Ejercicio 4:

Implementar un algoritmo que ordene una lista de elementos donde siempre el elemento del medio de la lista contiene antes que él en la lista la mitad de los elementos menores que él. Explique la estrategia de ordenación utilizada.

Ejemplo de lista de salida

7	3	2	8	5	4	1	6	10	9
---	---	---	---	---	---	---	---	----	---

La idea de este algoritmo es reorganizar los números alrededor del valor central (el pivote) para cumplir la regla (a la izquierda de él deben estar la mitad de los elementos menores al pivote) sin que este se mueva de su lugar.

Para lograrlo, hace lo siguiente:

1. Toma el número que está exactamente en el medio de la lista original, el cual será nuestro "pivote" y su posición no va a cambiar.
2. Busca todos los números de la lista que sean más chicos que el pivote y divide ese grupo en dos mitades iguales. Manda una mitad a la izquierda del pivote y la otra a la derecha.
3. Como el pivote no se puede mover, el lado izquierdo tiene que mantener la misma cantidad de elementos que tenía al principio. Para lograrlo, el algoritmo usa los números más grandes (los que son mayores o iguales al pivote) para rellenar los lugares que faltan a la izquierda. Los grandes que sobran los manda al lado derecho.
4. Finalmente, vuelve a unir todo: el lado izquierdo ya completo, el pivote en el medio, y el lado derecho.

Ejercicio 5:

Implementar un algoritmo **Contiene-Suma(A,n)** que recibe una lista de enteros A y un entero n y devuelve True si existen en A un par de elementos que sumados den n. Analice el costo computacional.

Realicé dos algoritmos para resolver el problema, cada uno con un costo computacional diferente: uno tiene una complejidad de $O(n^2)$, mientras que el otro logra reducirla a $O(n)$.

Algoritmo	Características	Costo computacional
contiene_suma	Recorre la lista mediante dos ciclos for, comparando cada elemento con los siguientes para encontrar un par cuya suma sea igual a n.	$O(n^2)$
contiene_suma2	Calcula el complemento que necesita cada elemento para alcanzar n y lo busca en un conjunto de elementos ya recorridos.	$O(n)$

Ejercicio 6:

Investigar otro algoritmo de ordenamiento como BucketSort, HeapSort o RadixSort, brindando un ejemplo que explique su funcionamiento en un caso promedio. Mencionar su orden y explicar sus casos promedio, mejor y peor.

Counting Sort es un algoritmo de ordenamiento que funciona contando la cantidad de ocurrencias de cada elemento en un arreglo. Utiliza esta información para construir el arreglo ordenado de manera eficiente. Funciona bien cuando el rango de valores posibles no es demasiado grande.

Ejemplo:

Supongamos que tenemos el siguiente arreglo:

[4, 2, 2, 8, 3, 3, 1]

Primero se cuenta cuántas veces aparece cada número:

- 1 aparece 1 vez.
- 2 aparece 2 veces.
- 3 aparece 2 veces.
- 4 aparece 1 vez.
- 5, 6 y 7 aparecen 0 veces.
- 8 aparece 1 vez.

Luego se recorren estos valores en orden y se reconstruye el arreglo utilizando las cantidades obtenidas:

[1, 2, 2, 3, 3, 4, 8]

- Caso promedio: Su complejidad es $O(n + k)$, ya que debe recorrer los elementos del arreglo y el rango de valores para realizar el conteo y construir el arreglo ordenado.
- Mejor caso: También es $O(n + k)$, porque aunque el arreglo ya esté ordenado o tenga pocos elementos repetidos, Counting Sort igualmente debe realizar el conteo y recorrer el rango de valores.
- Peor caso: Es $O(n + k)$. Se produce cuando el rango de valores k es muy grande en relación con la cantidad de elementos n , ya que se necesita recorrer y almacenar información para todo el rango.

Ejercicio 7:

A partir de las siguientes ecuaciones de recurrencia, encontrar la complejidad expresada en $\Theta(n)$ y ordenarlas de forma ascendente respecto a la velocidad de crecimiento. Asumiendo que $T(n)$ es constante para $n \leq 2$. Resolver 3 de ellas con el método maestro completo: $T(n) = aT(n/b) + f(n)$ y otros 3 con el método maestro simplificado: $T(n) = aT(n/b) + n^c$

a. $T(n) = 2T(n/2) + n^4$

$$a = 2, b = 2, f(n) = n^4$$

$$\log_2(2) = 1$$

$$1 + 3 = 4 \Rightarrow \varepsilon = 3$$

$$f(n) = \Omega(n^{1+3}) = \Omega(n^4), \quad 3 > 0$$

$$2f\left(\frac{n}{2}\right) \leq cf(n), \quad c < 1$$

$$2\left(\frac{n}{2}\right)^4 \leq c \cdot n^4$$

$$\frac{2n^4}{16} \leq c \cdot n^4$$

$$8n^4 \leq c \cdot n^4$$

Por ejemplo, para $c = \frac{1}{2}$, se cumple la condición.

Por lo tanto:

$$T(n) = \Theta(f(n)) = \Theta(n^4)$$

b. $T(n) = 2T(7n/10) + n$

$$a = 2, b = 10/7, f(n) = n = n^1$$

$$\log_{(10/7)}(2) = 2.0600$$

$$1 < 2.0600$$

$$\varepsilon = 1$$

$$f(n) = O\left(n^{(\log_{(10/7)}(2)-\varepsilon)}\right)$$

$$f(n) = O(n^{(2.0600-1)})$$

$$f(n) = O(n^{1.0600})$$

Por lo tanto, se cumple la condición del caso 1 del Teorema Maestro.

$$T(n) = \Theta\left(n^{(\log_{(10/7)}(2))}\right)$$

$$T(n) = \Theta(n^{2.0600})$$

c. $T(n) = 16T(n/4) + n^2$

$$a = 16, b = 4, f(n) = n^2$$

$$\log_4(16) = 2$$

$$n^{(\log_4(16))} = n^2$$

$$f(n) = \Theta(n^{(\log_4(16))})$$

$$f(n) = \Theta(n^2)$$

Por lo tanto, se cumple la condición del caso 2 del Teorema Maestro.

$$T(n) = \Theta(n^{(\log_4(16))} \log n)$$

$$T(n) = \Theta(n^2 \log n)$$

$$T(n) = \Theta(n^2 \log n)$$

d. $T(n) = 7T(n/3) + n^2$

$$a = 7, b = 3, c = 2$$

$$\log_3(7) \approx 1.7712$$

$$2 > 1.7712$$

Entonces estamos en el caso 3:

$$T(n) = \Theta(n^c)$$

Resultado: $\theta(n^2)$

e. $T(n) = 7T(n/2) + n^2$

$$a = 7, b = 2, c = 2$$

$$\log_2(7) \approx 2.8074$$

$$2 < 2.8074$$

Entonces estamos en el caso 1:

$$T(n) = \theta(n^{(\log_2(7))})$$

$$T(n) = \theta(n^{2.8074})$$

Resultado: $\theta(n^{2.8074})$

f. $T(n) = 2T(n/4) + \sqrt{n}$

$$a = 2, b = 4, c = 1/2$$

$$\log_4(2) = 1/2$$

Como:

$$c = \log_4(2)$$

estamos en el caso 2:

$$T(n) = \theta(n^c \log n)$$

$$T(n) = \theta(\sqrt{n} \log n)$$

Resultado: $\theta(\sqrt{n} \log n)$

A tener en cuenta:

1. Usen lápiz y papel primero

2. ~~No se puede utilizar otra Biblioteca mas alla de algo1.py y linkedlist.py~~

3. Hacer un análisis por cada algoritmo implementado del caso mejor, el caso peor y una perspectiva del caso promedio.

Ejercicio 4:

Mejor caso: Ocurre si la lista ya cumple la condición desde el principio. Sin embargo, como el algoritmo no tiene forma de saberlo antes de empezar, igual debe recorrer todos los elementos para separarlos. Tiempo: **$O(n)$** .

Peor caso: Ocurre cuando todos los elementos son estrictamente menores o mayores al pivote (el pivote es el extremo). No realiza cálculos extra ni se traba, simplemente hace las mismas separaciones. Tiempo: $O(n)$.

Caso promedio: En cualquier lista desordenada al azar, el comportamiento es exactamente igual. Siempre debe recorrer la lista completa para filtrar y luego unir las partes, por lo que su comportamiento es lineal: $O(n)$.

Ejercicio 5:

Algoritmo contiene_suma -> $O(n^2)$

- **Mejor caso:** ocurre cuando los dos primeros elementos de la lista suman n . El algoritmo encuentra la solución en la primera comparación, por lo que realiza una cantidad constante de operaciones: $O(1)$.
- **Peor caso:** ocurre cuando no existe ningún par que sume n o cuando el par se encuentra entre los últimos elementos. En este caso debe comparar prácticamente todos los pares posibles: $O(n^2)$.
- **Caso promedio:** en promedio, deberá recorrer una parte considerable de la lista y comparar múltiples pares, por lo que su comportamiento es $O(n^2)$.

Algoritmo contiene_suma2 -> $O(n)$

- **Mejor caso:** ocurre cuando el complemento del primer elemento ya se encuentra en el conjunto. En ese caso, termina rápidamente: $O(1)$.
- **Peor caso:** ocurre cuando no existe ningún par que sume n . Debe recorrer todos los elementos de la lista: $O(n)$.
- **Caso promedio:** normalmente deberá recorrer una parte de la lista antes de encontrar un par o determinar que no existe, por lo que su comportamiento es $O(n)$.